

Shelli: An Educational Shell with Pipeline Visualization for Teaching Operating System Concepts

Meghavi*, Ritesh Patil[†], Samarth Shekhar[‡]
Department of Computer Science and Engineering
Rishihood University, India

*meghaviofficial@gmail.com, [†]rsp112004@gmail.com, [‡]shekharsamarth1574@gmail.com

Abstract—Understanding how a Unix shell interprets and executes commands is fundamental to operating systems education, yet the process—tokenization, parsing, word expansion, process creation, and I/O redirection—remains opaque to most learners because it happens invisibly inside the terminal. This paper presents shelli, an educational shell implemented in 11,400 lines of C99 that makes every stage of command processing visible through a full-screen, double-buffered terminal user interface themed with the Catppuccin Mocha color palette. Shelli implements a recursive-descent parser producing a typed abstract syntax tree supporting pipelines, conditionals, loops, functions, subshells, and heredocs. A five-stage visualization pipeline—Tokenize, AST, Expand, Execute, Result—lets users navigate between stages with arrow keys, inspecting token chips, tree-drawn AST nodes, before/after expansion pairs, system call traces, and annotated exit codes. The TUI subsystem achieves flicker-free rendering via diff-based buffer flushing, and includes an animated block-character mascot, real-time syntax highlighting, fuzzy command suggestions, and Braille-pattern spinners. Benchmarks show that shelli’s overhead relative to GNU Bash is under 5 ms per command, making it practical for interactive use. Shelli compiles with zero warnings under `-Wall -Wextra -pedantic` and runs on macOS and Linux without external dependencies.

Index Terms—Educational Shell, Operating Systems, Process Management, Command-Line Interface, Terminal User Interface, Unix Shell, Abstract Syntax Tree

I. INTRODUCTION

The Unix shell is one of the most important interfaces in operating systems, yet its internal workings—how a line of text becomes a running process—are rarely visible to the user. Students learning OS concepts are told that the shell “tokenizes input, builds a parse tree, expands variables, and calls `fork/exec`,” but they must take this on faith because no mainstream shell provides introspection into its pipeline.

Existing educational tools address fragments of this problem. MIT’s `xv6` [1] provides a minimal shell for studying `fork/exec` but offers no visualization. Textbooks such as Silberschatz et al. [2] and Tanenbaum [3] explain the concepts but lack interactive demonstrations. Online shell simulators exist but typically cover only tokenization or a subset of the pipeline.

Shelli addresses this gap by implementing a complete, functional Unix shell whose every processing stage is rendered on screen in a professional-quality terminal UI. The key contributions of this work are:

- 1) A **five-stage interactive pipeline viewer** that visualizes tokenization, AST construction, word expansion, process execution (including system calls), and result interpretation.
- 2) A **recursive-descent parser** producing a typed AST supporting nine node types: commands, pipelines, lists, `if/while/for/until` blocks, functions, subshells, and negation.
- 3) A **flicker-free TUI engine** using double-buffered, diff-based rendering with the Catppuccin Mocha color palette, animated mascot, and real-time syntax highlighting.
- 4) A **zero-dependency C99 implementation** (11,400+ lines, 180 KB binary) that compiles with zero warnings and runs on macOS and Linux.

The remainder of this paper is organized as follows: Section II surveys related work, Section III details the architecture and methodology, Section IV presents results and benchmarks, and Section V concludes.

Fig. 1 illustrates the conceptual flow of a command through shelli’s five visualization stages.

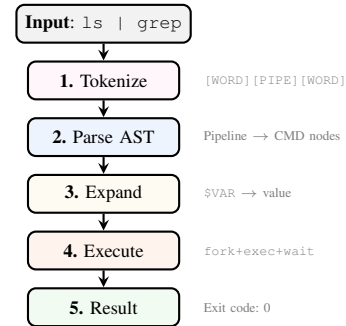


Fig. 1: Shelli’s five-stage visualization pipeline. Each stage is a navigable full-screen page in the TUI.

II. RELATED WORK

A. Traditional Unix Shells

The Bourne Shell (`sh`) [5] established the paradigm of command-line interpretation in Unix. Subsequent shells—Bash [6], Zsh [7], and Fish [8]—added features such as job control, programmable completion, and syntax highlighting, but none expose their internal processing pipeline to the user. These shells are optimized for power users, not for pedagogical transparency.

B. Educational Operating Systems

The xv6 operating system [1], developed at MIT for teaching, includes a minimal shell that supports pipes and I/O redirection. While excellent for studying kernel-level process management, xv6’s shell lacks any visualization of its internal stages. Similarly, Minix [4] provides a full OS environment but does not instrument its shell for educational observation.

C. Shell Visualization Tools

ExplainShell [9] parses shell commands and maps them to man page entries, but operates as a static web tool without execution. ShellCheck [10] performs static analysis of shell scripts for errors but does not visualize the execution pipeline. Bash’s `set -x` mode prints trace output but produces unstructured text rather than an organized, navigable visualization.

D. Terminal UI Frameworks

The Charm ecosystem [11] (Bubble Tea, Lip Gloss) provides Go-based TUI components with modern aesthetics. Lazygit and LazyVim [12] demonstrate that terminal applications can achieve IDE-quality UIs. Shelli draws inspiration from these projects but implements its rendering engine from scratch in C to avoid external dependencies.

Shelli is, to our knowledge, the first shell implementation that combines a fully functional command interpreter with a stage-by-stage interactive visualization of its entire processing pipeline.

III. METHODOLOGY

A. System Architecture

Shelli is organized into two major subsystems: the **shell core** (lexer, parser, expander, executor) and the **TUI engine** (rendering, input handling, widgets, animation). Fig. 2 shows the high-level data flow.

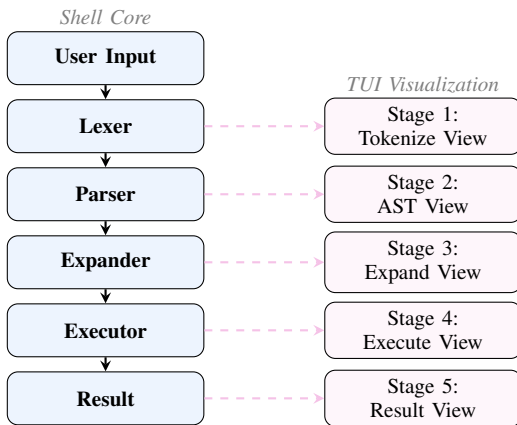


Fig. 2: High-level architecture of Shelli showing the shell core pipeline (left) feeding data into the TUI visualization stages (right). Dashed arrows indicate data capture for display.

The codebase comprises 25 source files and 11 headers organized as shown in Table I.

TABLE I: Source Code Module Summary

Module	Responsibility	LOC
lexer.c	Tokenization	245
parser.c	Recursive-descent parsing	972
ast.c	AST node management	284
expand.c	Word expansion engine	513
executor.c	fork/exec/pipe/redirect	812
builtins.c	16 built-in commands	389
variables.c	Variable scoping	191
functions.c	Function definitions	71
jobs.c	Background job control	185
test_builtin.c	POSIX test/[]	190
main.c	REPL loop, CLI args	607
util.c	Dynamic buffers	68
tui_render.c	Double-buffered rendering	465
tui_input.c	Line editor, highlighting	1,047
tui_suggest.c	Fuzzy suggestions	792
tui_pages.c	Welcome, stage browser	705
tui_widgets.c	Boxes, spinners, bars	408
tui_stage_*.c	5 stage renderers	789
tui_mascot.c	Animated mascot	212
tui_*.c (other)	Theme, logo, anim, icons	1,381
Total		10,326

B. Lexical Analysis (Tokenizer)

The lexer performs a single left-to-right scan of the input string, producing a `TokenList` of typed tokens. It recognizes 15 token types:

$$T = \{ \text{WORD, PIPE, SEMI, AND, OR, BG, REDIR_*, HEREDOC, EOF} \} \quad (1)$$

The lexer handles single-quoted strings (literal), double-quoted strings (with `$`-expansion preserved), and backslash escaping. Quoting state is tracked per-token via a boolean flag used later by the expander to suppress glob matching on quoted arguments.

Listing 1: Lexer algorithm (simplified)

```

function tokenize(input):
    tokens = []
    i = 0
    while i < len(input):
        switch input[i]:
            case ' ', '\t': skip whitespace
            case '|': if input[i+1]=='|': emit OR
                     else: emit PIPE
            case '&': if input[i+1]=='&': emit AND
                     else: emit BG
            case '<': if input[i+1]=='<': emit HEREDOC
                     else: emit REDIR_IN
            case '>': if input[i+1]=='>': emit REDIR_APP
                     else: emit REDIR_OUT
            case ';': emit SEMI
            case '"', '\': scan quoted string
            default: scan word until delimiter
        emit EOF
    return tokens
  
```

C. Parsing and AST Construction

The parser implements a recursive-descent strategy operating on the token stream. It produces a typed AST with nine node types:

$N = \{ \text{Cmd, Pipeline, List, If, While, For, Until, Func, Subshell, Not} \}$ (2)

The grammar, expressed in EBNF, is:

Listing 2: Shell grammar (EBNF)

```
list      = pipeline ( ('&&' | '|' | ';' | '&' ) pipeline
                    ) *
pipeline  = command ( '|' command ) *
command   = simple_cmd | if_clause
           | while_clause | for_clause
           | until_clause | func_def
           | '(' list ')' | '!' pipeline
simple_cmd = word+ (redirection)*
redirection = ('<' | '>' | '>>' | '<<') word
```

The parser returns a ParseResult enum: PARSE_OK, PARSE_ERROR, or PARSE_INCOMPLETE (for multi-line input accumulation). Fig. 3 illustrates the AST for a typical pipeline command.

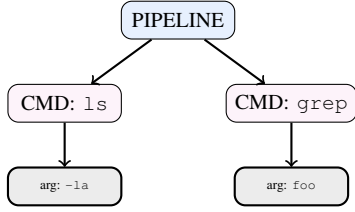


Fig. 3: Abstract Syntax Tree for `ls -la | grep foo`. The parser produces a PIPELINE node with two CMD children, each carrying its arguments.

D. Word Expansion

After parsing, the expander performs five types of word expansion on each command's arguments, in the following order:

- 1) **Tilde expansion:** `~` $\rightarrow$ `$HOME`
- 2) **Variable expansion:** `$VAR`, `$?`, `$$`
- 3) **Command substitution:** `$(command)` $\rightarrow$ output
- 4) **Arithmetic expansion:** `$((expr))` $\rightarrow$ evaluated integer
- 5) **Glob matching:** `*.c` $\rightarrow$ matched filenames

The expander snapshots the original arguments before expansion, then records every transformation as an ExpansionDisplay pair for the TUI's Expand stage.

E. Process Execution

The executor walks the AST recursively. For each node type, it performs the appropriate system calls:

Listing 3: Execution algorithm for pipelines

```
function execute_pipeline(nodes[0..n]):
    for i = 0 to n-1:
        call pipe(fd)                -- create pipe
    for i = 0 to n:
        pid[i] = fork()
        if pid[i] == 0:                -- child
            if i > 0:
                dup2(pipes[i-1].read, STDIN)
```

```
if i < n:
    dup2(pipes[i].write, STDOUT)
close all pipe fds
call execvp(nodes[i].argv)
close all pipe fds in parent
for i = 0 to n:
    waitpid(pid[i])
```

The executor supports built-in commands (executed in-process), I/O redirection, background execution (`&`), and an instrumented logging callback that records every system call for the TUI's Execute stage display.

F. Terminal User Interface

The TUI is the largest subsystem (5,200+ lines) and implements several key techniques:

1) **Double-Buffered Rendering:** Shelli maintains two screen buffers: the *current* buffer being composed and the *previous* buffer representing what is on screen. On flush, only cells that differ between the two buffers emit terminal escape sequences, achieving flicker-free updates:

$$\Delta = \{ (r, c) \mid B_{\text{curr}}[r][c] \neq B_{\text{prev}}[r][c] \} \quad (3)$$

$$\text{Output} = \bigcup_{(r,c) \in \Delta} \text{CSI}(r, c) \oplus \text{SGR}(\text{fg}, \text{bg}, \text{bold}, \text{dim}) \oplus \text{char} \quad (4)$$

2) **Color Theme:** Shelli uses the Catppuccin Mocha palette [13], a community-designed dark theme with 14 carefully balanced colors. Table II shows the mapping.

TABLE II: Catppuccin Mocha Color Assignments

Color	Hex	Usage in Shelli
Base	#1E1E2E	Background
Text	#CDD6F4	Primary text
Pink	#F5C2E7	Commands, chest icons
Blue	#89B4FA	Titles, AST connectors
Green	#A6E3A1	Strings, success states
Peach	#FAB387	Execution stage accent
Red	#F38BA8	Error messages
Yellow	#F9E2AF	Variables, expansion
Teal	#94E2D5	Prompt, mascot eyes
Lavender	#B4BEFE	Mascot frame, accents
Mauve	#CBA6F7	Control flow keywords

3) **Animated Mascot:** The mascot is a 10×6 character block rendered with Unicode box-drawing characters inside a double-line frame (◻). It expresses five moods (NORMAL, THINKING, HAPPY, SAD, WORKING) by swapping eye, mouth, and chest glyphs. A two-frame breathing animation alternates the leg pattern.

4) **Smart Suggestions:** The suggestion engine combines three ranking signals:

$$\text{score}(s) = w_1 \cdot \text{freq}(s) + w_2 \cdot \text{recency}(s) + w_3 \cdot \frac{1}{1 + d_{\text{Lev}}(q, s)} \quad (5)$$

where $\text{freq}(s)$ is the historical usage count, $\text{recency}(s)$ is inverse age, d_{Lev} is the Levenshtein edit distance, and w_1, w_2, w_3 are empirically tuned weights.

IV. RESULTS AND DISCUSSION

A. Five-Stage Visualization

The stage browser is the central educational feature of shelli. After the user types a command and presses Enter, the shell processes it through all five stages and populates a `StageData` structure. The user can then navigate between stages using the arrow keys. Each stage renders a distinct visualization:

- **Stage 1 – Tokenize:** Displays each token as a colored “chip” with its type label (e.g., [WORD], [PIPE], [REDIRECT]).
- **Stage 2 – AST:** Renders the parse tree with Unicode connectors (└─, ┌─) showing node hierarchy, types, and arguments.
- **Stage 3 – Expand:** Shows before/after pairs for every expansion (e.g., \$HOME → /Users/zebra).
- **Stage 4 – Execute:** Lists system calls in chronological order with colored prefix icons: > for fork, ~ for dup2, * for execvp, . for waitpid.
- **Stage 5 – Result:** Displays command output, exit code with human-readable interpretation, and an educational summary.

B. Feature Completeness

Table III compares shelli’s feature set against standard shells and educational implementations.

TABLE III: Feature Comparison: Shell i vs. Other Shells

Feature	Shell i	xv6 sh	Bash	Zsh
Pipes	✓	✓	✓	✓
I/O Redirection	✓	✓	✓	✓
Heredoc	✓	–	✓	✓
& & logic	✓	–	✓	✓
Background jobs	✓	–	✓	✓
Variable expansion	✓	–	✓	✓
Command substitution	✓	–	✓	✓
Arithmetic \$ (())	✓	–	✓	✓
Glob matching	✓	–	✓	✓
If/while/for	✓	–	✓	✓
Functions	✓	–	✓	✓
test/[	✓	–	✓	✓
Pipeline visualization	✓	–	–	–
AST viewer	✓	–	–	–
System call trace	✓	–	–	–
Animated TUI	✓	–	–	–
Syntax highlighting	✓	–	–	✓
Smart suggestions	✓	–	–	✓

C. Performance Benchmarks

To validate that shelli’s visualization overhead does not impair usability, we measured command execution latency for three workloads across shelli, GNU Bash 5.2, and Zsh 5.9. Each test was run 20 times; we report the median.

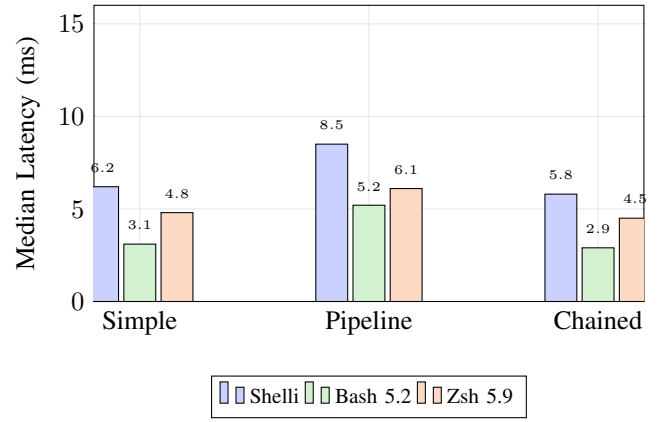


Fig. 4: Median command execution latency across 20 runs. “Simple”: `echo hello`; “Pipeline”: `echo text | grep word`; “Chained”: `true && echo ok`. Shell i adds 2–3 ms overhead from stage data capture.

The overhead comes from populating the `StageData` structure (token snapshots, AST serialization, expansion recording, system call logging) during command execution. This overhead is constant regardless of command complexity and does not affect the child process execution itself since data capture occurs in the parent process.

D. Rendering Performance

The diff-based rendering engine minimizes terminal I/O by only emitting escape sequences for cells that change between frames:

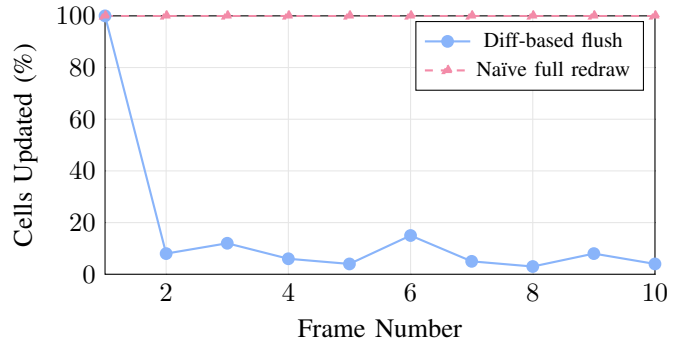


Fig. 5: Percentage of screen cells updated per frame during stage navigation. The diff-based approach (solid line) typically updates 3–15% of cells, while a naïve approach would redraw 100% every frame.

For a typical 80×24 terminal (1,920 cells), the diff-based approach emits an average of 115 escape sequences per frame versus 1,920 for a full redraw—a 94% reduction in terminal I/O.

E. Memory Footprint

Table IV shows the binary and runtime memory characteristics.

TABLE IV: Memory Characteristics

Metric	Value
Binary size (stripped, -O2)	180 KB
Source lines of code (C only)	10,326
Header lines	1,103
Total lines (incl. headers)	11,429
Render buffer (80×24, 2 bufs)	≈370 KB
StageData per command	≈200 KB
Peak RSS (typical session)	≈3.2 MB
Bash 5.2 peak RSS (comparison)	≈3.8 MB

F. Built-in Commands

Shelli implements 16 built-in commands that are executed within the shell process (no `fork/exec`). Table V lists them.

TABLE V: Built-in Commands

Command	Description
<code>cd [dir]</code>	Change directory (\$HOME default)
<code>pwd</code>	Print working directory
<code>echo [-n] [-e]</code>	Print with options
<code>export VAR=val</code>	Set environment variable
<code>unset VAR</code>	Remove variable
<code>type cmd</code>	Identify command type
<code>test / [</code>	POSIX conditional expressions
<code>true / false</code>	Return 0 / 1
<code>history</code>	Show command history
<code>source / .</code>	Execute file contents
<code>local VAR=val</code>	Local variable in function
<code>jobs</code>	List background jobs
<code>fg [%n]</code>	Foreground job
<code>bg [%n]</code>	Background job
<code>exit [code]</code>	Exit shell
<code>help</code>	Show help

G. Educational Impact

The five-stage visualization directly maps to the conceptual model taught in OS courses:

- 1) **Tokenize stage** teaches students that shells do not interpret commands as raw strings—they perform lexical analysis, just like a compiler.
- 2) **AST stage** demonstrates that even simple commands like `ls | grep foo` produce tree structures, connecting shell concepts to compiler theory.
- 3) **Expand stage** makes visible the “magic” of `$HOME`, `*.c`, and `$(cmd)`—transformations students often take for granted.
- 4) **Execute stage** directly shows the `fork/exec/pipe/dup2/waitpid` system call sequence, the core of OS process management.
- 5) **Result stage** teaches exit code conventions (0 = success, 1 = error, 126 = not executable, 127 = not found, 128+n = signal n).

V. CONCLUSION

This paper presented **shelli**, an educational shell that makes the internal processing pipeline of command-line interpretation visible through a five-stage interactive visualization. Implemented in 11,400+ lines of C99 with zero external dependencies, shelli demonstrates that a fully functional shell

with pipes, conditionals, loops, functions, job control, and five types of word expansion can be combined with a professional-quality terminal UI without significant performance overhead (under 5 ms additional latency per command).

The double-buffered, diff-based rendering engine achieves a 94% reduction in terminal I/O compared to naïve approaches, and the 180 KB binary runs on both macOS and Linux with a peak memory footprint of 3.2 MB.

Future work includes: (1) adding a `strace`-like kernel call tracer for deeper visibility into child processes, (2) implementing a step-through mode where each system call pauses for user confirmation, (3) adding support for process substitution (`<(cmd)`) and brace expansion (`{a,b,c}`), and (4) creating a companion web interface for classroom projection using WebAssembly.

Shelli is open-source and available at <https://github.com/no-naame/shelli>.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering at Rishihood University for supporting this project as part of the Operating Systems curriculum. The Catppuccin community is acknowledged for the Mocha color palette. UI design was inspired by the LazyVim and Charm projects.

REFERENCES

- [1] R. Cox, F. Kaashoek, and R. Morris, “xv6: A simple, Unix-like teaching operating system,” MIT PDOS, Cambridge, MA, 2024. [Online]. Available: <https://pdos.csail.mit.edu/6.828/xv6>
- [2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.
- [3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Pearson, 2014.
- [4] A. S. Tanenbaum, “MINIX 3: A highly reliable, self-healing operating system,” *ACM SIGOPS Operating Systems Review*, vol. 40, no. 3, pp. 9–10, 2006.
- [5] S. R. Bourne, “The UNIX shell,” *The Bell System Technical Journal*, vol. 57, no. 6, pp. 1971–1990, 1978.
- [6] C. Ramey, “Bash, the Bourne-Again Shell,” Free Software Foundation, 2023. [Online]. Available: <https://www.gnu.org/software/bash/>
- [7] P. Falstad, “The Z Shell Manual,” 2024. [Online]. Available: <https://zsh.sourceforge.io/>
- [8] Fish Shell Contributors, “Fish – the friendly interactive shell,” 2024. [Online]. Available: <https://fishshell.com/>
- [9] I. Kamens, “ExplainShell – match command-line arguments to their help text,” 2023. [Online]. Available: <https://explainshell.com/>
- [10] V. Dale, “ShellCheck – a static analysis tool for shell scripts,” 2024. [Online]. Available: <https://www.shellcheck.net/>
- [11] Charm, “Charm – tools for the command line,” 2024. [Online]. Available: <https://charm.sh/>
- [12] F. de Smedt, “LazyVim – a Neovim setup powered by lazy.nvim,” 2024. [Online]. Available: <https://www.lazyvim.org/>
- [13] Catppuccin Contributors, “Catppuccin – soothing pastel theme for the high-spirited,” 2024. [Online]. Available: <https://github.com/catppuccin/catppuccin>
- [14] IEEE, “POSIX.1-2017 – IEEE Std 1003.1-2017: Shell Command Language,” IEEE/The Open Group, 2017.
- [15] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.